

LUCRAREA NR. 9

INSTRUCȚIUNI CONCURENTE

1. Scopul lucrării

Informațiile din această lucrare vin în completarea celor din lucrarea precedentă. Sunt prezentate pe larg instrucțiunile concurente existente în VHDL, împreună cu aplicațiile lor principale și cu exemple de utilizare bogat documentate și comentate. Vor fi analizate: instrucțiunea **block**, apelul concurent de procedură, instrucțiunea concurentă **assert**, instrucțiunea concurentă de asignare de valori semnalelor, instrucțiunea de instanțiere a unei componente și instrucțiunea **generate**.

2. Considerații teoretice

2.1 Descrierea concurență

Există o diferență importantă între noțiunile de *concurență* și *parallelism*, care se cuvine a fi subliniată.

Instrucțiunile paralele se execută în funcție de prioritățile atribuite de sistem, fără nici un alt punct de sincronizare în afară de cele programate explicit (prin construcții specifice de genul: primitivele P și V, semafoare, monitoare etc.). Acesta este paralelismul întâlnit în anumite limbaje de programare. Fondul acestui mecanism este nedeterminist și noțiunea de „timp real” este adeseori prezentă „în fundal”.

În cazul instrucțiunilor concurente, ordinea de execuție este oarecare. Aceasta permite punerea în aplicare a unui paralelism real, dar noțiunea de timp real dispare: *simularea nu se efectuează în timp real*.

În concluzie, putem spune că paralelismul limbajelor de programare cunoscute este un paralelism *virtual* în timp *real*, în vreme ce concurența din VHDL este un paralelism *real* în timp *virtual* (timpul de simulare).

Descrierea unui sistem hardware se face în mod firesc sub o formă concurentă. Un asemenea sistem se proiectează ca un ansamblu de module, de flux de date sau de algoritmi (în funcție de nivelul de descriere ales) care interacționează, funcționând în paralel.

Aceste funcții sunt descrise de instrucțiuni concurente care se execută în mod asincron; în plus, ordinea în care sunt scrise instrucțiunile nu are nici o influență asupra ordinii lor de execuție.

„Puterea” unui limbaj de descriere hardware poate fi evaluată în funcție de bogăția setului său de instrucțiuni concurente. În cazul VHDL, acesta este deosebit de complet și de universal. Instrucțiunile sunt utilizate pentru descrierea comportamentului diferitelor module din componența circuitului. Prin urmare, ele se vor regăsi la nivelul unităților de proiectare prin intermediul cărora se realizează descrierea: entitățile și arhitecturile.

Într-o descriere concurentă, variabilele cunoscute din limbajele de programare clasice nu-și mai au locul, interacțiunea (transportul de informație între procese) fiind modelată numai prin intermediul semnalelor.

2.2 Instrucțiunea *block*

Blocul reprezintă o modalitate de reunire a unui set de instrucțiuni concurente pentru a le face să aibă acces la o serie de declarații locale, invizibile din orice altă parte a restului descrierii. El permite de asemenea scrierea de instrucțiuni de asignare *gardate* sau ierarhizarea descrierii.

Cele trei funcții principale ale unui bloc sunt următoarele:

- încapsularea declarațiilor;
- utilizarea instrucțiunilor de asignare *gardate*;
- suport pentru ierarhizarea proiectului.

Aceste trei funcții nu se exclud reciproc: un bloc poate foarte bine să-și asume toate cele trei roluri deodată.

Sintaxa generală a unui bloc este următoarea:

```
etichetă: block {(condiție_de_gardă)}  
    {antet_parametri_genèrici_și_porturi}  
    ...  
    ... Declarații locale  
    ...  
begin  
    ...  
    ... Instrucțiuni concurente  
    ...  
end block {etichetă};
```

Condiția de gardă trebuie să fie o expresie booleană. Ea poate fi adeseori exprimată printr-o frază în limbaj natural, de genul „când semnalul X ia o anumită valoare”.

Antetul opțional indică valorile (pentru parametrii generici) și semnalele (pentru porturi) pe care blocul le importă din mediul exterior.

2.2.1 Funcția de încapsulare a declarațiilor

Firește, partea declarativă a unui bloc nu este vizibilă decât în interiorul blocului respectiv. Declarațiile permise în interiorul unui bloc sunt următoarele:

- declarație de sub-program;
- corp de sub-program;
- declarație de tip;
- declarație de sub-tip;
- declarație de constantă;
- declarație de semnal;
- declarație de fișier;
- declarație de alias;
- declarație de componentă;
- declarație de atribut;
- specificație de atribut;
- declarație de grup;
- specificație de deconectare;
- specificație de configurație;
- clauza **use**.

De remarcat interdicția declarării *variabilelor* locale blocului (datorită faptului că natura blocului este concurentă).

2.2.2 Utilizarea instrucțiunilor gardate

Această posibilitate răspunde unei nevoi care apare în mod frecvent la proiectarea circuitelor logice. O mare parte a circuitului este *sincronă*, ceea ce înseamnă că toate asigările de semnale sunt supuse unei condiții de tact. În general, un circuit sincron este un circuit în cadrul căruia unul sau mai multe evenimente se pot produce în același timp. De exemplu, pentru un circuit sincronizat pe frontul ascendent al semnalului de tact, fiecare asignare de valoare unui semnal ar trebui să se scrie astfel:

```
A <= expresie when (CLOCK = '1' and not CLOCK'STABLE);
```

Rezultatul acestei asignări poate fi văzut în schema din figura 9.1. Asignarea rezultatului expresiei semnalului A nu se efectuează decât pe frontul ascendent al semnalului de tact CLOCK.

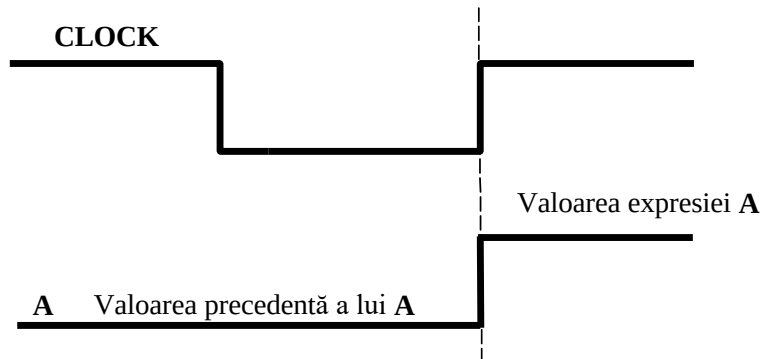


Figura 9.1 Asignarea semnalului A pe frontul ascendent al semnalului CLOCK

Observație

Atributul STABLE returnează valoarea booleană TRUE atunci când semnalul vizat nu și-a modificat valoarea în pasul curent al simulării. Condiția de mai sus înseamnă deci că semnalul de tact are valoarea '1' și nu este stabilă (tocmai s-a modificat) în pasul curent al simulării. Atributul STABLE este prezentat în detaliu în lucrarea nr. 5.

Ar fi laborios să se scrie în VHDL descrieri care să repete aceeași condiție de un mare număr de ori. *Garda* unui bloc oferă însă posibilitatea „factorizării” acestor condiții. Așadar, scrierea blocului gardat (condiția de gardă este întotdeauna booleană) este echivalentă cu cea a unui bloc fără gardă în care se definește un semnal boolean numit GUARD, care ia valoarea expresiei de gardă și în care instrucțiunile de asignare se efectuează condiționat.

În exemplul de mai jos se prezintă două blocuri echivalente:

```
BLOC1: block (CLOCK = '1' and not CLOCK'STABLE)
```

```

begin
    S1 <= guarded DATA1 after 10 ns;
    S2 <= guarded DATA2 after 15 ns;
end block BLOC1;

BLOC2: block
signal GUARD: BOOLEAN;
begin
    GUARD <= CLOCK = '1' and not CLOCK'STABLE;
    S1 <= DATA1 after 10 ns when GUARD else S1;
    S2 <= DATA2 after 15 ns when GUARD else S2;
end block BLOC2;

```

Semnalul GUARD poate fi declarat explicit și, cu toate acestea, poate fi recunoscut ca declanșator al asignării gardate (cu ajutorul cuvântului cheie **guarded**). În plus, fie că este sau nu declarat, acest semnal poate face obiectul unei asignări.

2.2.3 Suport pentru proiectarea ierarhică

Blocul este unitatea de bază echivalentă a structurării în VHDL.

În practică, rolul blocului în cadrul structurării este echivalent cu cel al procesului în cazul simulării. Orice altă formă de structurare a proiectului nu este, de fapt, decât un artificiu echivalent cu un bloc, căci în cele din urmă orice construcție VHDL se reduce la un bloc.

De exemplu, o componentă instanțiată de către o entitate (ansamblul specificație de entitate-arhitectură referit într-o configurație) este echivalentă cu două blocuri imbricate.

Următorul exemplu este extras din Manualul de referință (LRM) al VHDL. Se dă un pachet PAC care exportă o procedură CHECKTIMING a cărei specificație poate fi scrisă după cum urmează:

```

package PAC is
    procedure CHECKTIMING (signal A, B: BIT; T: TIME);
end PAC;

```

Fie următoarea pereche alcătuită din specificația de entitate X și din arhitectura Y:

```

use WORK.PAC.all; -- Referirea pachetului PAC
entity X is

```

```

    port(P1, P2: inout BIT);
    constant DELAY: TIME := 1 ms;
begin
    CHECKTIMING(P1, P2, 2*DELAY);
end X;
architecture Y of X is
    signal P3: BIT;
begin
    P3 <= P1 after DELAY;
    P2 <= P3 after DELAY;
    B: block
        ...
        begin
            ...
        end block;
end Y;

```

și fie o componentă COMP a cărei declarație este următoarea:

```

component COMP
    port (A, B: inout BIT);
end component;

```

și pe care o instanțiem într-o componentă C plecând de la perechea specificație de entitate X - arhitectură Y. Această entitate având porturile P1 și P2 și nu A și B, trebuie să specificăm corespondențele:

```

for C: COMP use entity X(Y) port map (P1 => A, P2 => B);

```

și în final urmează să specificăm configurația componentei C (în care S1 și S2 sunt două semnale care trebuie conectate la această instanță):

```

C: COMP port map (A => S1, B => S2);

```

Această instrucțiune va fi echivalentă cu următoarele două blocuri imbricate:

```

C: block
    port(A, B: inout BIT);
    port map (A => S1, B => S2);
end block;

```

```

begin
  X: block
    port(P1, P2: inout BIT);
    port map (P1 => A, P2 => B);
    constant DELAY: TIME := 1 ms;
    signal P3: BIT;
    begin
      CHECKTIMING(P1, P2, 2*DELAY);
      P3 <= P1 after DELAY;
      P2 <= P3 after DELAY;
      B: block
        ...
        begin
          ...
        end block;
    end block X;
end block C;

```

Partea declarativă a blocului înglobat (X) conține atât declarațiile specificației de entitate cât și cele ale arhitecturii Y. De asemenea, partea rezervată instrucțiunilor blocului înglobat (X) conține atât instrucțiunile specificației de entitate cât și cele ale arhitecturii Y.

În practică, această ierarhie tradusă sub formă de blocuri este interesantă mai ales pentru programatorii de compilatoare / simulatoare VHDL. În cazul modelării, ierarhia va fi rareori tradusă prin instrucțiuni **block** (se lasă acest lucru pe seama compilatorului), preferându-se noțiunea de *componentă*.

2.3 Apelul concurent de procedură

Apelul concurent de procedură are aceeași sintaxă ca și apelul secvențial de procedură (singura diferență care apare este posibilitatea specificării unei etichete opționale).

Întrucât ne aflăm în domeniul concurent, parametrii procedurii nu pot fi decât semnale sau constante (NU și variabile!). Un apel de procedură se efectuează întotdeauna prin indicarea numelui procedurii urmat, între paranteze, de lista parametrilor de apel (apelul poate fi pozițional, prin nume sau combinat). Opțional, numele acestei proceduri poate fi precedat de către o etichetă.

Să luăm exemplul unei proceduri care verifică dacă un semnal A respectă un timp de prepoziționare (*set-up*) TS (valoare declarată ca o

constantă) față de frontul ascendent al unui alt semnal, B. Semnalele A și B vor fi transmise în mod **in** (este o verificare – valorile acestor semnale nu vor suferi modificări). Apelul concurrent al procedurii se va putea scrie astfel:

```
VERIF1: VERIF_SETUP (A, B, TS);
```

Un apel concurrent de procedură va fi tradus în domeniul secvențial printr-un proces care înglobează apelul secvențial corespondent, urmat de o instrucțiune **wait** care va monitoriza semnalele de mod **in** și **inout** date ca parametri. Cu alte cuvinte, procedura va fi apelată la fiecare eveniment care va apărea pe parametrii săi care sunt semnale de intrare. Instrucțiunea **wait** este indispensabilă pentru evitarea unei bucle infinite, în care procesul respectiv ar intra în lipsa ei. În cazul în care printre parametrii săi nu există nici un semnal de mod **in** sau **inout**, instrucțiunea **wait** nu va avea nici o condiție: aceasta înseamnă că procedura nu va fi apelată decât o singură dată, la inițializare.

În cazul exemplului precedent, procesul echivalent se rezumă la:

```
VERIF1: process
begin
    VERIF_SETUP(A, B, TS);
    wait on A, B;
end process VERIF1;
```

Așadar, numărul de procese create este egal cu numărul de apeluri ale procedurii respective. Următoarea porțiune de cod creează trei procese:

```
VERIF1: VERIF_SETUP(A, B, TS);
VERIF2: VERIF_SETUP(C, D, TS_CD);
VERIF3: VERIF_SETUP(A, B, TS);
```

Aceste procese vor „trăi” până la sfârșitul simulării.

Utilizarea imediată a apelului concurrent de procedură survine atunci când trebuie să creăm mai multe procese implementând aceiași algoritmi, dar care fie nu se aplică acelorași semnale, fie au constante diferite. Algoritmul se scrie deci o singură dată sub formă de procedură (de exemplu,

într-un pachet) și fiecare instrucțiune de apel va crea un proces specific pe baza parametrilor transmiși.

Printr-o scriere adecvată a procedurii apelate, putem obține două efecte foarte interesante:

- Dacă procedura se încheie printr-o instrucțiune **wait** fără condiții, ea se va executa o singură dată, la inițializare;
- Dacă scriem o procedură sub formă de buclă infinită având grijă să introducem o instrucțiune **wait** (sensibilă la semnalele de intrare, de mod **in** și **inout**) imediat înainte de sfârșitul buclei, vom suprapune astfel iterația procesului peste iterativitatea derulării procedurii respective.

De exemplu, prin apelul concurrent al procedurii:

```

procedure VERIF_SETUP (signal S, REF: in BIT; TIMP: TIME) is
begin
    loop
        ...    -- Algoritm
        ...
        wait on S, REF;
    end loop;
end VERIF_SETUP;

```

se obține același comportament ca și la apelul aceleiași proceduri fără buclă și fără instrucțiune **wait**, cu o singură diferență: apelul procedurii VERIF_SETUP nu este efectuat decât o dată și prin urmare obiectele eventual declarate în partea declarativă a acestei proceduri (în cazul de față nu există nici unul) sunt remanente. Acest aspect este deosebit de important în cazul oricărei proceduri care gestionează starea internă a unui automat finit.

2.4 Instrucțiunea concurentă *assert*

Instrucțiunea are o sintaxă identică (cu excepția etichetei, care este opțională) cu cea a instrucțiunii secvențiale **assert**.

```

{etichetă:}    assert    condiție    {report    mesaj}    {severity
nivel_de_eroare};

```

La fel ca în cazul instrucțiunii secvențiale, această instrucțiune are ca efect monitorizarea unei condiții și emiterea unui mesaj în cazul în care aceasta este *falsă*. De fapt, ea reprezintă o simplă versiune simplificată de scriere a următorului proces:

```
{etichetă:} process  
begin  
assert condiție {report mesaj}{severity nivel_de_eroare};  
wait on lista_semnalelor_care_aparîn_condiție;  
end process {etichetă};
```

Observație

Este important de reținut că acest proces echivalent nu are listă de sensibilitate, ci conține o instrucțiune secvențială **wait** după instrucțiunea secvențială **assert**. Prin urmare, chiar dacă lista semnalelor care apar în condiție este vidă, procesul nu se va putea repeta în buclă.

Instrucțiunea **assert** se poate afla într-o specificație de entitate sau într-o arhitectură. În cadrul unei specificații de entitate, ea permite de exemplu controlarea validității semnalelor prezente la porturi sau a valorilor parametrilor generici.

Asemănarea cu instrucțiunea secvențială **assert** nu trebuie să ascundă puterea acestei instrucțiuni. Într-adevăr, caracterul concurent și iterativ al procesului face ca monitorizarea condiției date să fie permanentă. Simpla scriere a unei instrucțiuni concurente **assert**, de exemplu pentru monitorizarea verificării timpului de prepoziționare (*set-up*) al unui bistabil, va crea un control permanent al acestei caracteristici.

La fiecare modificare a valorii unui semnal care ar putea modifica valoarea condiției, se va efectua un test și se va genera, eventual, mesajul de eroare specificat. Acest caracter de „viață proprie” al controlului este foarte util, cu atât mai mult cu cât absența ieșirilor acestui proces (singura „ieșire” este – eventual – mesajul specificat) evită complicarea inutilă a simulării.

Observație

Scrierea procesului echivalent al acestei instrucțiuni ascunde o capcană! În cazul în care condiția nu depinde de nici un semnal, lista

semnalelor care intervin în cadrul condiției este vidă și aceasta va fi verificată doar o singură dată, la inițializare, și niciodată după aceea.

De exemplu, prin utilizarea condiției `NOW < TIMP_MAX` (unde `TIMP_MAX` este o constantă și `NOW` este funcția care returnează timpul curent al simulării), nu vom ajunge niciodată la sfârșitul simulării.

Dacă dorim să testăm o condiție care nu utilizează nici un semnal, atunci vom fi nevoiți să scriem un proces care va conține instrucțiunea *secvențială* **assert** și va avea o listă de sensibilitate adecvată.

2.5 Instrucțiunea concurentă de asignare de valori semnalelor

Asignarea de valori semnalelor poate îmbrăca o *formă condițională* sau o *formă selectivă*. Fiecăreia dintre aceste forme îi corespunde câte o formă secvențială, cea a instrucțiunii de asignare conținută în procesul echivalent.

Sintaxa *formei condiționale* este următoarea:

```
{etichetă:} nume_sau_agregat <= {guarded}
    formă_de_undă_1 when condiție_booleană_1 else
    formă_de_undă_2 when condiție_booleană_2 else
    ...
    formă_de_undă_n;
```

Presupunând că se omite opțiunea **guarded**, această formă se traduce în următorul proces echivalent (deci, în instrucțiuni secvențiale):

```
if condiție_booleană_1 then
    nume_sau_agregat <= {transport} formă_de_undă_1;
elsif condiție_booleană_2 then
    nume_sau_agregat <= {transport} formă_de_undă_2;
...
else
    nume_sau_agregat <= {transport} formă_de_undă_n;
end if;
```

Această formă se numește *forma condițională echivalentă*.

Sintaxa *formei selective* este următoarea:

```
{etichetă:} with expresie select
    nume_sau_agregat <= {guarded}{transport}
        formă_de_undă_1 when alegere_1,
```

```
formă_de_undă_2 when alegere_2,  
...  
formă_de_undă_n when alegere_n;
```

La fel ca în cazul formei condiționale, presupunând că se omite opțiunea **guarded**, această formă se traduce în următorul proces echivalent (*forma selectivă echivalentă*):

```
case expresie is  
  when alegere_1 =>  
    nume_sau_agregat <= {transport} formă_de_undă_1;  
  when alegere_2 =>  
    nume_sau_agregat <= {transport} formă_de_undă_2;  
    ...  
  when alegere_n =>  
    nume_sau_agregat <= {transport} formă_de_undă_n;  
end case;
```

După cum se poate remarca, există două opțiuni comune acestor două forme:

- Opțiunea **transport**, care a fost deja prezentată în cadrul lucrării nr. 3, specifică un model de propagare non-inerțial (toate impulsurile sunt transmise). Dacă apare în cadrul instrucțiunii concurente de asignare, această opțiune va fi prezentă la nivelul fiecărei asignări secvențiale a formelor echivalente;
- Opțiunea **guarded** arată că asignarea nu va fi executată decât dacă semnalul GUARD de tip BOOLEAN (declarat implicit într-un bloc gardat sau explicit de către proiectant) are sau este pe cale să primească valoarea TRUE. Eventual se va genera o instrucțiune de deconectare. O asignare care utilizează această opțiune se numește *asignare gardată*.

Observație

La declararea sa, un semnal poate fi clasificat ca având tipul registru (cuvântul cheie **register**) sau magistrală (cuvântul cheie **bus**). Atunci este vorba despre un semnal gardat (*guarded signal*). Aceste calificări, foarte

apropiate de preocupările fizice, hardware, permit descrierea comportamentului unui semnal la deconectarea sa.

Așadar, în cazul unei instrucțiuni concurente de asignare de semnal, este posibilă apariția unuia dintre următoarele patru cazuri:

1. Semnal ne-gardat, obiect al unei asignări ne-gardate. Asignarea concurentă se reduce la forma echivalentă (condițională sau selectivă);
2. Semnal ne-gardat, obiect al unei asignări gardate. Asignarea concurentă se reduce la următoarea condiție:

```
if GUARD then
    formă echivalentă (condițională sau selectivă)
end if;
```

3. Semnal gardat, obiect al unei asignări ne-gardate. Acest caz este interzis: compilatorul sau simulatorul VHDL va genera o eroare. Un semnal gardat nu poate fi asignat decât prin intermediul unei instrucțiuni gardate;
4. Semnal gardat, obiect al unei asignări gardate. Asignarea concurentă devine foarte puternică în acest caz, căci ea va gestiona implicit și deconectarea semnalului, în cazul în care asignarea (care este gardată) nu va fi executată (cu alte cuvinte, dacă condiția de gardă este falsă).

```
if GUARD then
    formă echivalentă (condițională sau selectivă)
else
    instrucțiune de deconectare
end if;
```

Instrucțiunea de deconectare îi asignează semnalului gardat un element de formă de undă nul după întârzierea indicată în specificația de deconectare a acestui semnal. Asignarea unui element de formă de undă nul este rezervată semnalelor gardate și se scrie astfel:

```
semnal_sau_agregat <= null after {expresie_de_tip_TIME};
```

Semnalul astfel asignat rămâne fără sursă („firul este întrerupt”) după un interval de timp dat. Această instrucțiune înlătură din pilotul semnalului tot ceea ce este datat „mai târziu” decât timpul dat în momentul specificării deconectării.

Observație

O arhitectură este un bloc (fără condiție de gardă). Într-un bloc fără condiție de gardă putem însă declara un semnal GUARD și apoi putem folosi instrucțiuni gardate. Așadar, exemplul de mai jos este perfect valabil:

```
architecture ARH1 of ENT1 is
  signal A: BIT;
  signal GUARD: BOOLEAN;
begin
  A <= guarded '1' after 10 ns;
end ARH1;
```

Așadar, este inutil să creăm un bloc în interiorul unei arhitecturi, dacă acest bloc (numit și *instrucțiune block*) va fi singura instrucțiune a respectivei arhitecturi.

2.6 Instrucțiunea de instanțiere a unei componente

2.6.1 Instanțierea

A *instanția* o componentă înseamnă a face o copie (*o instanță*) a unei componente deja declarate („modelul”) și de a o personaliza pentru a da curs unei necesități particulare. De exemplu, dacă am declarat deja o componentă INVERSOR, atunci putem instanția o componentă INV1, apoi o componentă INV2 etc. Sintaxa instrucțiunii este următoarea:

```
etichetă: numele_componentei_model
{corespondența parametrilor generici}
{corespondența porturi efective / porturi locale modelului};
```

Această instrucțiune permite descrierea unui modul al circuitului ca fiind o instanță a unui model. Modelul este definit printr-o declarație de componentă. Se pot preciza următoarele informații:

- *Valoarea parametrilor generici ai componente.* Acest aspect reprezintă adaptarea componente la nevoile de modelare curente;

- *Corespondența dintre porturile formale ale componentei model și porturile efective, cele pe care le vom conecta în circuitul nostru actual. Există posibilitatea de a nu conecta toate porturile modelului, caz în care porturile efective corespunzătoare nu vor exista (vor fi „în aer”). În cazul porturilor de intrare (de mod **in**), acestora li se vor asina valorile implicite prevăzute eventual pentru ele.*

În exemplul de mai jos, se obțin două componente, C1 și C2, care asigură interconectarea semnalelor A, B, C și D de tip BIT.

```
signal A, B, C, D: BIT;

-- Declararea componentei INVERSOR
-- INTRARE și IEȘIRE sunt porturile modelului, iar A, B, C și
-- D sunt porturile efective ale circuitului nostru
component: INVERSOR
    port(INTRARE: in BIT; IEȘIRE: out BIT);
end component;

-- Componenta INVERSOR va fi instanțiată de două ori
C1: INVERSOR port map (INTRARE => A, IEȘIRE => B);
C2: INVERSOR port map (INTRARE => C, IEȘIRE => D);
```

2.6.2 Diferența dintre bloc și componentă

Ce diferențe există între un bloc și o instanță a unei componente?

Când trebuie creat un bloc și când trebuie instanțiată o componentă?

Nu există nici o diferență fundamentală între un bloc și o instanță a unei componente: după cum am văzut, o componentă este un bloc pentru că ea este imaginea unei entități. Ambele construcții sunt folosite pentru ierarhizarea *descrierii structurale*.

Un bloc poate fi văzut ca o componentă deja instanțiată. În acest sens, el reprezintă o variantă simplificată de scriere: crearea unui bloc evită recurgerea la referirea unei entități, la declararea unei componente, la configurarea și apoi la instanțierea sa. Deși este ceva mai greu de utilizat în practică, componenta are un mare avantaj față de bloc: ea este reutilizabilă.

Instanțierile componentelor se fac cu ajutorul unor parametri (porturi) diferite. Această instanțiere este un mecanism foarte puternic, pentru că permite omiterea anumitor porturi și mai ales schimbarea dinamică (prin configurație) a entității care va fi luată drept model.

Să remarcăm că blocul are o altă funcție, efemeră în viața ciclului de proiectare: el permite, într-o fază intermediară, crearea și testarea rapidă a ceea ce – mai târziu – va deveni o componentă. Acest rol nu este deloc neglijabil pentru proiectant!

În concluzie, este foarte util să alegem o structurare în blocuri a proiectului – dacă urmărim o scriere cât mai concisă, o proiectare rapidă sau dacă excludem orice reutilizare a codului. Dimpotrivă, structurarea în componente va aduce gradul de generalitate necesar unor aplicații complexe.

2.7 Instrucțiunea *generate*

Această instrucțiune permite elaborarea iterativă sau condițională a liniilor de cod sursă VHDL; ea NU efectuează o execuție condițională sau iterativă a instrucțiunilor înglobate.

Cele două forme (condițională și iterativă) ale acestei instrucțiuni sunt:

```
-- Forma condițională
etichetă: if condiție_booleană generate
    ...
    Secvență de instrucțiuni concurente
    ...
end generate {etichetă};
-- Forma iterativă
etich: for nume_param_generare in interval_discret generate
    ...
    Secvență de instrucțiuni concurente
    ...
end generate {etich};
```

Semantica formei condiționale este foarte simplă: instrucțiunile concurente înglobate nu există decât dacă, la elaborare, condiția respectivă este adevărată (TRUE).

Extrem de utilă, forma iterativă creează un număr de ansambluri de instrucțiuni înglobate egal cu numărul de elemente cuprins în intervalul discret dat (eventual nici unul). Pentru fiecare ansamblu de instrucțiuni, parametrul generării (un identificator) este înlocuit, oriunde apare, cu valoarea iterației.

Prezentăm în continuare un exemplu care utilizează ambele forme ale acestei instrucțiuni.

Să presupunem că dorim să conectăm o serie de N inversoare, ieșirea unuia fiind conectată la intrarea următorului. Vom folosi în continuare exemplul din secțiunea 2.6.1, în care am instanțiat două componente INVERSOR. În acest caz, problema este diferită, deoarece nu se cunoaște dinainte numărul de componente necesare.

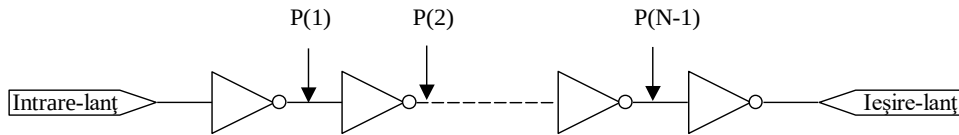


Figura 9.2 Lanț de N inversoare: entitatea LANȚ_DE_INVERSOARE

N va fi un parametru generic transmis entității LANȚ_DE_INVERSOARE care trebuie proiectată. Specificația acestei entități este următoarea:

```
entity LANȚ_DE_INVERSOARE is
  generic (N: INTEGER);
  port (INTRARE_LANȚ: in BIT; IEȘIRE_LANȚ: out BIT);
end LANȚ_DE_INVERSOARE;
```

Entitatea conține două porturi: INTRARE_LANȚ și IEȘIRE_LANȚ. Mai rămâne de descris arhitectura sa asociată, știind că numărul de inversoare din lanț este N .

```
architecture STRUCTURALĂ of LANȚ_DE_INVERSOARE is
  -- Declararea unui semnal care memorează cele N-1 noduri
  -- interne ale lanțului de inversoare
  signal P: BIT_VECTOR (1 to N-1);
```

```

--Declaraarea componentei INVERSOR
component INVERSOR
    port (INTRARE: in BIT; IEȘIRE: out BIT);
end component;

-- Toate instanțele componentei INVERSOR precedente vor fi
-- configurate pentru a utiliza specificația entității cu
-- același nume(INVERSOR) asociată arhitecturii COMP.
for all: INVERSOR use entity INVERSOR(COMP);
begin

-- Instanțierea primului inversor: forma condițională a
-- instrucțiunii generate
PRIMUL_INVERSOR: if N >= 1 generate
    INVERSOR_1: INVERSOR port map (INTRARE_LANȚ, P(1));
end generate PRIMUL_INVERSOR;

-- Instanțierea inversoarelor intermediare: forma iterativă a
-- instrucțiunii generate
ALTE_INVERSOARE: for I in 1 to N-2 generate
    INVERSOR_I: INVERSOR port map (P(I), P(I+1));
end generate ALTE_INVERSOARE;

-- Instanțierea ultimului inversor: forma condițională a
-- instrucțiunii generate
ULTIMUL_INVERSOR: if N >= 2 generate
    INVERSOR_N: INVERSOR port map (P(N-1), IEȘIRE_LANȚ);
end generate ULTIMUL_INVERSOR;
end STRUCTURALĂ;

```

Mai există încă o posibilitate de specificare a arhitecturii: vom utiliza numai forma iterativă a instrucțiunii **generate** și vom memora N+1 noduri. Cele două noduri suplimentare sunt legate la intrare și la ieșire prin intermediul a două instrucțiuni de asignare concurentă de semnal.

```

architecture STRUCTURALĂ_2 of LANȚ_DE_INVERSOARE is
-- Declaraarea unui semnal care memorează cele N+1 noduri
signal P: BIT_VECTOR (1 to N+1);
component INVERSOR
    port (INTRARE: in BIT; IEȘIRE: out BIT);
end component;

```

```
for all: INVERSOR use entity INVERSOR (COMP);
begin
-- Instanțierea tuturor componentelor
INVERSOARE: for I in 1 to N generate
    INVERSOR_K: INVERSOR port map (P(I), P(I+1));
end generate INVERSOARE;
-- Primul nod se conectează la intrarea lanțului
P(1) <= INTRARE_LANȚ;
-- Al (N+1)-lea nod se conectează la ieșirea lanțului
IEȘIRE_LANȚ <= P(N+1);
end STRUCTURALĂ_2;
```

Acest exemplu ne permite să facem două observații importante:

1. Parametrul N trebuie să fie cunoscut în momentul elaborării: prin urmare, N poate fi o constantă sau un parametru generic;
2. În cazul de față, configurarea inversoarelor se face „din mers” pentru toate componentele (**for all...**).

3. Desfășurarea lucrării

- 3.1 Se va implementa un DEMULTIPLEXOR 1 la 8 descriind structura sa internă sub formă de porți logice, unde fiecare poartă logică va constitui un bloc intern.
- 3.2 Se va proiecta și implementa un registru de deplasare universal BARREL SHIFTER care poate efectua deplasări la stânga sau la dreapta de maximum 3 poziții binare. Se va studia oportunitatea utilizării instrucțiunii concurente assert și a blocurilor gardate.
- 3.3 Pentru implementarea bistabililor din componența registrului de deplasare universal BARREL SHIFTER de la punctul anterior, se vor instanția componente BISTABIL_D.
- 3.4 Se va testa exemplul LANȚ_DE_INVERSOARE din lucrare.
- 3.5 Se va implementa un sumator de numere pe N biți folosind instrucțiunea **generate**.